QC SKILL · RECONSTRUCTION SPECIFICATION

qc-hardening

A 14-pass production audit that drives residual concrete failure scenarios to zero or explicit deferral, without changing functional behavior. Requires qc-core.

Canonical source of truth is the **SKILL.md** tree in this repository. This PDF reconstructs that tree for porting. Do not treat this PDF as a second design. Do not vendor a specialization without **qc-core**.

Role

REQUIRED: load qc-core first. Laws, ledger schema 1.1, verdict, `.qc-profile.json`, Discovery+Verify, severity, deferred vocabulary, CLEAN lists, and commits live there. This skill is the 14-pass specialization. It does not reimplement those invariants.

Drive residual **concrete failure scenarios** — especially under realistic call sequences, dual/public paths, and live external/adapter boundaries — to zero or explicit deferral, without changing functional behavior. Mechanical passes 1–11 remove noise; Carmack (12) and Chaos (13) prove absences tools miss; Maintainability (14) converts the evidence into a release judgment. Clean Carmack across all modules plus a green Chaos suite is the expected terminal state.

Hardening owns the runtime proof of sequences and live paths. Coherence owns static contracts and canonical forms. Harvest Carmack violated_invariant values into profile invariants[]. If maintainability debt cannot be fixed locally, defer it (needs-architecture); do not rewrite architecture.

Fourteen passes (do not collapse or reorder)

Pass	Name	Notes
1	Correctness	Composition-primary: sequences, dual-path parity, field population, classification completeness, I/O coercion on every ingest path. Isolated function bugs are residual.
2	Security	Secrets, injection, dependency audit.
3	Error handling and resilience	Boundaries, resource lifecycle, validation.
4	Type safety and contracts	Type checker + linter. Quick-check eligible after streaks.
5	Concurrency and async	Skip if no concurrency signals. Quick-check eligible.
6	Performance and scale	Concrete n, no micro-optimize. Quick-check eligible.
7	Logging and observability	Quick-check eligible.
8	Test coverage gaps	Writes tests only. 8a before deep; 8b after 12–13.
9	Documentation accuracy	Fix docs to match code, not the reverse.
10	Backward compatibility	Skip if no public API. Quick-check eligible.
11	Env, config, build	Quick-check eligible.
12	Carmack review	Five adversarial questions. EXAMINED is the Carmack manifest on .qc-profile.json.
13	Chaos monkey	In-repo tests only. Production-shaped sequences. No Game Days, no production chaos.
14	Maintainability	Release judgment is primary (maps to READY / READY_WITH_DEBT / NOT_READY). Letter grades are supporting evidence.

Quick-check eligible after 3 clean streaks and no new code in scope: 4, 5, 6, 7, 10, 11. Never quick-check 1, 2, 3, 8, 9, 12, 13, 14.

Carmack EXAMINED (Pass 12)

Each EXAMINED module requires answers to:

- What invariant makes this correct?
- What happens when my assumptions are wrong?
- What sequence of valid operations produces an invalid state?
- If this runs a million times, what breaks?
- What would make this code wrong tomorrow?

Or the compact form {invariant, assumptions, sequence_risk}, all substantive. Priority: untested live external and adapter paths first. Finding trigger names a combination, not "this function is wrong." Escalation: grep the pattern, fix all instances, write pass_debt[] if a mechanical pass should have caught the combination.

Chaos (Pass 13)

Writes tests only (tests/test_chaos.py). Each production-shaped test is an explicit hypothesis. Blast radius is the test file. Required sequences when the surface exists: wrong order of valid steps; partial success then failure; empty-then-large; double-submit. Do not run production chaos, Game Days, or invent recovery/monitoring.

Pass 14 release judgment

Primary output maps onto the qc-core enum: READY (release-ready as-is), READY_WITH_DEBT (releasable with deferred maintainability debt), NOT_READY (not yet releasable). Letter grades (Consistency / Maintainability / Overall) are supporting evidence. Every sub-A grade names a concrete change that would force whole-system context.

P0 policy

Hardening uses **presence** P0: any P0, even fixed, is NOT_READY. That is the original human-review gate. The suite rollup also blocks on any P0 in any skill ledger.

Reexam

Compute the reexam set with qc-core partition.py --reexam --since <sha> --profile .qc-profile.json. Neighbors that sit in hot_spots[] / pass_debt[] are mandatory composition re-examination even if they did not change.

What not to do

- Do not add features, refactor architecture, or change public signatures / parameter defaults.
- Do not change test assertions to match buggy code without independent proof the test is wrong.
- Do not manufacture findings. Clean pass = clean pass.
- Do not copy qc-finding.schema.json or verify_ledger.py into this skill; call qc-core.